

MIPSolvers 原生 LP 内核 (lp_kernel)

数学模型与面向千万维问题的严苛数据结构与算法效率审计

代码级静态审计报告 (所有论断附 file:line 引用)

2026 年 8 月 1 日

摘要

本报告直接依据源代码, 记录三方面内容: (i) MIPSolvers 仓库 LP 内核 (`src/engine/kernel/lp_kernel/` 及其线性代数后端 `src/engine/kernel/linear_algebra/` 中的 `HFactorBackend`) 所求解的数学模型——标准形变换、Ruiz 平衡、修订对偶/原始单纯形的完整迭代数学、稳定化与反循环机制、不可行证书; (ii) 以通用 LP 求解器须求解上千万维度 ($m = n = 10^7$ 、 $\text{nnz}(A) \approx 10^8$) 问题为基准, 对其数据结构与算法效率进行的刻意严苛的审计; (iii) 对”占位符/伪实现”的系统性普查——被声明、被精心编写、看起来复杂, 但实际上从未被调用、或在数学上不构成其所声称之物的代码。每项论断附 `file:line` 引用, 行号对应报告日期当日的工作区状态。本审计为静态审计: 未做运行时剖析, 复杂度陈述均源自代码本身。

总体结论 (先说结论)。与”大量占位符”的怀疑相反, `native_dual/` 内核的热路径是真实的: 它是一个教科书规格的有界变量修订对偶单纯形——懒堆 CHUZR、超稀疏打包 FTRAN/BTRAN、Harris 两遍 BFRT、Goldfarb-Reid DSE 权重、Forrest-Tomlin 更新、带秩修复的 INVERT、区间算术 Farkas 证书。但代码中确实存在三类与用户怀疑相符的实质问题: (a) 约 290 行长双精度”精确验证”机器 (`pivot_evidence`、`row_solve_consistent`、`refine_ftran`) 在全仓库**零调用**, 是纯粹的死代码; (b) 若干以”精确性”为名的验证趟 (每次 INVERT 3-4 趟 $O(\text{nnz}(A))$ 长双精度残差、冷启动 m 次带验证 BTRAN 的精确 DSE 初始化) 在 10^7 维下各自单独即致命; (c) 约束矩阵被完整存储三份、每次 LU 重构再全量拷贝一次, 索引全部为 32 位 `int`。该内核是按 $m \sim 10^3-10^4$ (IEEE-118、NETLIB、MIPLIB 根节点 LP) 设计与验证的; 在当前形态下它无法生存于千万维问题, §6 给出逐条定量证据与改造清单。

目录

1 范围、方法与文件清单	2
1.1 范围	2
1.2 方法	2
1.3 文件清单与角色	2
2 数学模型	3
2.1 原始 LP 与标准形变换	3
2.2 Ruiz 平衡缩放	4

目录	2
2.3 有界变量修订对偶单纯形 (Phase II)	4
2.4 对偶 Phase I (齐次坐标)	5
2.5 原始单纯形阶段 (Phase I/II 与清理)	6
2.6 稳定化: 成本扰动与工作成本平移	6
2.7 反循环: Zobrist 签名与塔布表	6
2.8 证书与 fail-closed 发布	7
2.9 HiGHS 委托路径的数学角色	7
3 数据结构评估	7
3.1 标准形 <code>StandardFormLP</code> : 矩阵存三份的源头	7
3.2 迭代状态 <code>State</code> : 稠密向量组 (合理但有账目)	8
3.3 稀疏向量 <code>IndexedVector</code> 与打包求解	8
3.4 CHUZR 堆与原始定价堆: 合格	9
3.5 反循环表: <code>std::map</code> 的不当选择与无界增长	9
3.6 LU/FT 后端 <code>HFactorBackend</code> : 真实但带全量拷贝	9
3.7 遗留与冗余字段	10
3.8 基准规模内存账目	10
4 算法效率评估	10
4.1 对偶 Phase II 单次主元: 真实的超稀疏实现	10
4.2 INVERT 与其”验证税”	11
4.3 冷启动初始化: 精确 DSE 是单独的灾难	12
4.4 原始单纯形阶段的成本轮廓	12
4.5 标准形与边界的增量维护: 合格	12
4.6 HiGHS 委托路径的成本	12
4.7 复杂度小结	13
5 占位符与缺陷清单	13
5.1 P 类: 死代码与虚假参数 (证据确凿)	13
5.2 Q 类: 数学缺口 (声称与实现不符)	14
5.3 R 类: 真实但仅小维有意义的实现	15
6 千万维可扩展性结论	15
6.1 致命项排序	15
6.2 必须修改的清单 (按投入产出排序)	16
6.3 最终结论	16
A 审计方法与可复核性	17

1 范围、方法与文件清单

1.1 范围

本次审计的组件是原生 LP 内核：

`src/engine/kernel/lp_kernel/`（对偶单纯形集成层 + `native_dual/` 原生内核）

`include/mipsolvers/engine/kernel/lp_kernel/`（公共头文件）

`src/engine/kernel/linear_algebra/hfactor_backend.cpp` 及其头文件（LU/FT 后端）

`native_dual/` 调用的 vendored HiGHS `HFactor`（`src/engine/kernel/linear_algebra/highs_factor/util/HFactor.cpp`）作为黑箱 LU 库对待，仅审计其包装层。IPM 内核（`kernel/ipm/`）、PDLP（`solver/native/lp/pdip_solver.cpp`）、KKT（`kernel/kkt/`）不在本次范围内。

1.2 方法

对全部 14 个源文件（合计约 9700 行）逐行阅读；所有关键论断随后按所引用行号复核。死代码判定使用全仓库（`src/` 与 `tests/`）符号引用搜索确认。报告区分三类陈述：**(V)** 经直接阅读所引代码验证（占绝大多数）；**(I)** 由代码结构推断，并明示推断依据；**(U)** 代码意图不明——显式标注。

1.3 文件清单与角色

表 1: LP 内核文件清单（行数为报告日工作区状态）

文件	行数	角色
<code>dual_simplex.cpp</code>	2486	公共集成层：HiGHS 委托路径、结果翻译、Farkas 导出、fail-closed 残差审计
<code>dual_simplex_api.cpp</code>	1081	标准形构建、割行追加、Ruiz 缩放、界/成本增量更新
<code>native_dual/solver.cpp</code>	1443	对偶单纯形主驱动：minor 迭代、阶段控制、清理
<code>native_dual/state.cpp</code>	1315	状态重建、审计、稳定化成本、循环签名
<code>native_dual/pricing.cpp</code>	1003	CHUZR 堆、BFRT/Harris、DSE/Devex 权重更新
<code>native_dual/primal.cpp</code>	839	原始单纯形阶段（Phase I/II、清理）
<code>native_dual/factor.cpp</code>	718	BasisFactor：带验证的求解证据、INVERT、FT 更新转发

文件	行数	角色
linear_algebra/hfactor_backend.cpp	773	vendored HiGHS HFactor 的包装 (LU、超稀疏求解、FT 更新)
native_dual/model.hpp	262	State/IndexVector/事务结构
native_dual/primal_pricing.hpp	228	原始 Devex/PSE 权重数学 (inline)
native_dual/factor.hpp	143	BasisFactor 接口
native_dual/certificate.cpp/.hpp	111	Farkas 乘子与区间证书
native_dual/state.hpp	93	状态函数声明
native_dual/pricing.hpp	33	定价函数声明
native_dual/primal.hpp	13	原始阶段入口声明
native_dual/certificate.hpp	11	证书入口声明
native_dual_core.hpp	149	状态/统计/结果类型与入口

一个塑造全篇报告的事实：**生产默认路径不是原生内核**。SimplexOptions::lp_kernel_backend 默认值为 LpKernelBackend::HiGHS (include/mipsolvers/engine/kernel/lp_kernel/dual_simplex.hpp:173, 枚举定义于 include/mipsolvers/engine/kernel/lp_kernel/backend.hpp:8-11), 此时 dual_simplex.cpp:1337-1694 把标准形 LP 完整委托给 vendored HiGHS 求解。原生 native_dual/ 内核须经 ExperimentalNative 显式选择——其注释自述“ExperimentalNative is an explicit development-only selection” (dual_simplex.hpp:172)。两条路径本报告都评估。

2 数学模型

2.1 原始 LP 与标准形变换

内核接受的公共模型 (LPModel) 为

$$\min_x c^\top x \quad \text{s.t.} \quad \text{lhs}_i \leq (Ax)_i \leq b_i \quad (i = 1, \dots, m_{\text{ineq}}), \quad A_{\text{eq}}x = b_{\text{eq}}, \quad \ell \leq x \leq u, \quad (1)$$

其中 lhs_i 可为 $-\infty$ (单边行), $\pm 10^{20}$ 为公共”无界”哨兵, 内核侧以 10^{19} 为识别阈值 (dual_simplex_api.cpp:23,26-32)。

build_standard_form_lp (dual_simplex_api.cpp:35-255) 把 (1) 变为内核的最大化、非负右端、带界变量标准形:

- (i) **下界平移**: $x = z + \bar{\ell}$, $\bar{\ell}_j = \ell_j$ (有限时), 否则 0 (dual_simplex_api.cpp:53-57)。所有变量平移到下界为 0。
- (ii) **行定向**: 每行乘以符号 $s_i \in \{+1, -1\}$ 使平移后右端 $\hat{b}_i \geq 0$ (dual_simplex_api.cpp:115-126)。
- (iii) **辅助列**: 'L' 行加松弛列 $+e_i$ (ranged 行取其宽度为上界, dual_simplex_api.cpp:66-99,238-243) ; 'G' 行加剩余列 $-e_i$ 和人工列 $+e_i$; 'E' 行加人工列 $+e_i$ (dual_simplex_api.cpp:134-158,189-199)。

得到标准形

$$\max_z c_{\max}^\top z \quad \text{s.t.} \quad \bar{A}z = \bar{b}, \quad 0 \leq z \leq \bar{u}, \quad \bar{u}_j \in (0, +\infty], \quad (2)$$

其中 $\bar{A} = SA$ (含辅助列), $c_{\max} = (-c_{\min}, 0)$, 常数项 $\text{obj_const} = c_{\min}^\top \bar{\ell}$ 。原问题最优值由 $\min c^\top x = \text{obj_const} - \max c_{\max}^\top z$ 恢复 (发布于 `dual_simplex.cpp:2405,1041,1606`)。标准形列数 $n = n_{\text{orig}} + n_{\text{slack}} + n_{\text{surplus}} + n_{\text{artificial}}$; 每个 'G'/'E' 行都有一个人工列, 因此逻辑基 (每行取其 slack/artificial 列) 恒为单位矩阵——这是冷启动基 (`native_dual/factor.cpp:24-38` 的 `logical_columns`)。

注记 1 (数学上的两个设计决定). (a) 人工列不是单纯形算法内部的 *Phase-I* 装置, 而是标准形的永久成员; *Phase II* 通过令其上界有效为 0/不可进入而冻结它们 (`make_phase_two_bounds`, `native_dual/state.cpp:180-195`: 人工列根本不进入 *enterable*)。委托给 *HiGHS* 时则显式固定为 0 (`dual_simplex.cpp:640-644`)。 (b) 目标恒为最大化; 最小化问题取负成本。所有对偶符号约定 (简约成本 $d = c - A^\top y$ 在 *max* 形式下的符号、行对偶发布时的符号翻转) 沿此展开 (`dual_simplex.cpp:501-504, 1675-1683`)。

2.2 Ruiz 平衡缩放

`ruiz_scale_standard_form` (`dual_simplex_api.cpp:775-846`) 执行至多 `rounds` 轮 (调用点固定为 10, `dual_simplex.cpp:1792`) 迭代平衡: 每轮令 $D_r = \text{diag}(\|A_i\|_\infty)^{-1/2}$, $D_c = \text{diag}(\|A_j\|_\infty)^{-1/2}$, 并就地施加 $\bar{A} \leftarrow D_r \bar{A} D_c$, $\bar{b} \leftarrow D_r \bar{b}$, $c_{\max} \leftarrow D_c c_{\max}$, $\bar{u} \leftarrow D_c^{-1} \bar{u}$, 累积因子记录于 `sf.row_scale/sf.col_scale`; 第 2 轮后若最大偏差 $< 10^{-2}$ 提前终止 (`dual_simplex_api.cpp:818`)。每轮成本 $O(\text{nnz}(\bar{A}))$, 且对行主、列主两份矩阵各做一遍 (`dual_simplex_api.cpp:821-831`) ——见 §3 的内存账目。

2.3 有界变量修订对偶单纯形 (Phase II)

记基指标集 β ($|\beta| = m$), $B = \bar{A}_\beta$ 。每个非基列 j 携带移动方向 $\mu_j \in \{\text{Down}, \text{Fixed}, \text{Up}\}$ (`Move`, `native_dual/model.hpp:32-38`), 表示其贴在有效上界或下界 (0)。对偶迭代维护:

$$x_\beta = B^{-1} \left(\bar{b} - \sum_{j \notin \beta} \bar{A}_{\cdot j} z_j \right), \quad z_j = \begin{cases} 0, & \mu_j = \text{Up}, \\ \bar{u}_j, & \mu_j = \text{Down}, \end{cases} \quad (3)$$

$$y = B^{-\top} (c_{\max})_\beta, \quad d = c_{\max} - \bar{A}^\top y. \quad (4)$$

对偶可行条件 (max 形式): $\mu_j = \text{Up} \Rightarrow d_j \leq \tau_D$, $\mu_j = \text{Down} \Rightarrow d_j \geq -\tau_D$ (容差 τ_D 为 `optimality_tol`, 判定见 `dual_infeasibility`, `native_dual/state.cpp:1192-1197`; μ_j 与 d_j 同号违规的组合判定见 `normalize_nonbasic_moves`, `native_dual/state.cpp:593-613`)。

一次 minor 迭代 (`minor_iteration`, `native_dual/solver.cpp:330-797`) 的数学步骤:

S1. CHUZR (`choose_leaving`, `native_dual/pricing.cpp:213-305`): 在原始不可行基行 (x_{β_p} 越出 $[0, \bar{u}_{\beta_p}]$ 超过 τ_P , 度量见 `primal_infeasibility`, `native_dual/state.cpp:1177-1190`) 中按 DSE 优度 $\text{merit}_i = v_i^2/w_i$ (v_i 为违例量, w_i 为边权重) 用懒惰版本化二叉堆进行 p ; 随即计算行对偶 $\pi_p = B^{-\top} e_p$ (打包 BTRAN)。

S2. PRICE(multiply_AT_indexed, native_dual/pricing.cpp:117-179): 计算主行 $\rho^\top = \pi_p^\top \bar{A}$ (ρ 为 n 维稀疏向量), 以邮戳 (epoch-stamp) 稠密累加器实现, 丢弃 $|\cdot| \leq 10^{-14}$ 的数值噪声项 (native_dual/pricing.cpp:156-178, 可用 MIPSOLVERS_PRICE_TIGHT=off 关闭)。

S3. BFRT/CHUZC (choose_entering_bfirt, native_dual/pricing.cpp:307-869): 对每个非基 j 计算带号主元 $\alpha_j = \text{side} \cdot \text{sign}(\mu_j) \cdot \rho_j$, 仅 $\alpha_j > 0$ 且通过点积误差证书 $\alpha_j > E_j$ 者可为主元; E_j 为 γ_k 放大的绝对点积界 (dot_error_bound, native_dual/pricing.cpp:87-105), 并有一个由 $\|\bar{A}_j\|_1$ 预计算的支配系数短路 (bfirt_error_coef, native_dual/state.cpp:1086-1106)。按 Harris 两遍法: 第一遍按断点 $\theta_j = \max(0, -\text{sign}(\mu_j)d_j)/\alpha_j$ 分组展开, 直到累计容量 $\sum \alpha_j(\bar{u}_j - 0)$ 覆盖离基违例 v_p (有界变量比测试 BFRT, native_dual/pricing.cpp:462-543); 第二遍在松弛步长界内取最大 α_j (native_dual/pricing.cpp:621-670)。被跳过的组对应的列执行界翻转 (bound flip), 其右端补偿 $\Delta \bar{b}$ 以稀疏事务形式物化并校验覆盖量 (native_dual/pricing.cpp:803-868)。

S4. FTRAN 与更新: $\delta = B^{-1}\bar{A}_q$ (打包 FTRAN, native_dual/solver.cpp:455-465); 主元恒等性要求列主元 δ_p 与行主元 ρ_q 一致; 原始步长 $t = (x_{\beta p} - \text{bound})/\delta_p$, 对偶步长 θ ; 就地更新 $x_\beta \leftarrow x_\beta - t\delta - \Delta_{\text{BFRT}}$ 、 $d \leftarrow d + \theta \text{side} \cdot \rho$ (native_dual/solver.cpp:642-758)。

S5. 基更新: Forrest-Tomlin 乘积形式更新 $B \leftarrow B + (\bar{A}_q - B e_p) e_p^\top$, 由 HFactorBackend::update_captured 转发 vendored HFactor (linear_algebra/hfactor_backend.cpp:734-766)。

S6. DSE 权重 (compute_dse_weights, native_dual/pricing.cpp:871-1000): Goldfarb-Reid 精确递推

$$w_i \leftarrow w_i - 2 \frac{\delta_i}{\delta_p} \varrho_i + \left(\frac{\delta_i}{\delta_p} \right)^2 w_p, \quad w_p \leftarrow \frac{w_p}{\delta_p^2}, \quad (5)$$

其中 $\varrho = B^{-1}\pi_p$ 需额外一次打包 FTRAN (native_dual/pricing.cpp:940-946); 带 1024ε 下界截断防负权重 (native_dual/pricing.cpp:980-990)。Devex 备选模式沿参考框架更新并在比值 > 9 时重启框架 (native_dual/pricing.cpp:882-938)。

INVERT 策略: 每 $\max(50, \min(200, \lfloor m/4 \rfloor))$ 次更新、或 HFactor 填充提示、或数值困难时重构 (native_dual/solver.cpp:976-986)。重构 = 重因子化 (带秩修复: 缺主元行以其逻辑列替换, factor.cpp:44-83、hfactor_backend.cpp:231-373) + 全量状态重建 (reconstruct, native_dual/state.cpp:519-591) + 界侧重分类 + 单边非基列的工作成本归零平移 (HiGHS HEkkDual 同款, native_dual/state.cpp:828-873)。

2.4 对偶 Phase I (齐次坐标)

若冷基无法选出原始界下对偶可行的端点, 内核进入对偶 Phase I (native_dual/solver.cpp:1279-1336): 构造锚点 x^0 使 $\bar{A}x^0 = \bar{b}$ (逻辑基下逐行求解, make_dual_phase_one_

anchor, native_dual/state.cpp:197-218, 并用长双精度残差验证, native_dual/state.cpp:278-286), 在齐次坐标 $z = x - x^0$ 中把单边列的有效区间设为 $[x_j^0, x_j^0 + 1]$ 、盒式列固定于 x_j^0 (make_dual_phase_one_bounds, native_dual/state.cpp:220-242)。Phase I 目标为剩余原始界对偶不可行度之和 (dual_phase_one_objective, native_dual/state.cpp:315-330); 其降为零后经 transition_dual_phase_one_to_two 恢复原始界进入 Phase II (native_dual/state.cpp:351-378)。这与 HiGHS 的对偶 Phase I 同构 (注释自述, native_dual/state.cpp:230-234)。

2.5 原始单纯形阶段 (Phase I/II 与清理)

run_primal_phase (native_dual/primal.cpp:431-837) 是完整的有界变量原始单纯形: 进入列堆定价 (优度 g_j^2/w_j , Devex 默认、PSE 可选, native_dual/primal.cpp:228-250, 334-362)、Harris 两遍原始比测试 (choose_leaving_harris, native_dual/primal.cpp:364-427)、盒式进入列的纯界翻转捷径 (native_dual/primal.cpp:544-569)、主元行/列恒等性校验 (native_dual/primal.cpp:652-667)、每 256 次更新的定期 INVERT (native_dual/primal.cpp:15, 825-833)。原始 Phase I 以人工变量和为目标 ($c_w = -1$, make_primal_phase_one_cost, native_dual/state.cpp:255-261), Phase I 最优值 $> \tau_P$ 时以 $\pi = B^{-T}c_\beta$ 构造 Farkas 证书判定原始不可行 (native_dual/solver.cpp:1178-1200)。其用途有三: 对偶 Phase I 失败的兜底、暖启动基非对偶可行时的修复 (allow_warm_primal_phase_one)、以及 Phase II 结束后去除成本扰动/平移后的**原始清理** (native_dual/solver.cpp:1040-1097)。

2.6 稳定化: 成本扰动与工作成本平移

两个 HiGHS 风格的退化处理机制: (a) **确定性成本扰动** (initialize_stabilized_cost, native_dual/state.cpp:735-790) : 仅当初始状态原始不可行时施加, 幅度 $5 \times 10^{-7} \cdot \max(1, \|c\|_\infty)^{1/4}$ (结构性列) 与 10^{-12} (逻辑列), 扰动相位由 splitmix64 哈希确定性生成 (deterministic_fraction, native_dual/state.cpp:24-31); (b) **工作成本平移** (cost shift): BFRT 大对偶步长会把单边非基列的简约成本带过零, 内核不平移界而是把该列工作成本精确归零 (native_dual/pricing.cpp:749-759; INVERT 时的同类平移见 native_dual/state.cpp:849-869)。两者都累积在 cost_shift/cost_perturbation 中, 终止前由强制的**原始成本清理**移除并重新审计 (restore_original_cost + reconstruct + audit, native_dual/solver.cpp:1040-1106)。

2.7 反循环: Zobrist 签名与塔布表

循环防护用双 64 位 Zobrist 集合哈希刻画 (基分配, 非基界侧) 状态 (compute_cycle_signature, native_dual/state.cpp:62-82), 关键工程点是每次主元只做 $O(1)$ 的 XOR 增量维护 (cycle_signature_apply_basis_swap/_move_toggle, native_dual/state.cpp:92-106; 全量重算曾占 IEEE-118 根 LP 求解时间的约 11%, 注释见 native_dual/state.cpp:40-46)。到达已见签名时, 对该签名下上一次离开/进入列对施加塔布 (寿命 $2m + 1$ 个定价纪元, native_dual/state.cpp:1051-1061); CHUZR/CHUZY 拒绝塔布候选, 全部候选

皆塔布时把该行加入行塔布，行塔布在候选耗尽时以 aspiration 方式提前释放（`native_dual/pricing.cpp:236-270`、`native_dual/state.cpp:985-997`）。这不是 Dantzig 反退化扰动的替代品，而是其上的第二道防线。

2.8 证书与 fail-closed 发布

- (i) **原始不可行 (Farkas) 证书**: CHUZC 在新鲜重构基上无候选时，以 $\pi = \pi_p$ （离基行对偶）构造区间证书：计算 $\pi^\top \bar{b}$ 与 $\pi^\top \bar{A}z$ 在 $z \in [0, \bar{u}]$ 上的可达区间 $[L, U]$ ，取间隔较大侧并要求 $\text{margin} > 1024\varepsilon \cdot \max(1, \text{绝对和})$ （`certificate_from_multiplier`, `native_dual/certificate.cpp:10-69`）。数学依据：若 $\pi^\top \bar{b} > \max_z \pi^\top \bar{A}z$ （或对称地小于下界），则 $\bar{A}z = \bar{b}$ 与界不可兼得。Phase I 人工目标版见 `phase_one_farkas_certificate`（`native_dual/certificate.cpp:87-98`）。
- (ii) **中断对偶界认证**: 时间/迭代限制触发时，内核恢复原始成本、重建状态、重做界侧分类并通过对偶可行审计；成功则发布的目标值是原始成本下的精确对偶目标，由弱对偶性给出 LP 最优值的严格界（`certify_interrupted_dual_bound`, `native_dual/solver.cpp:799-819`；导出见 `dual_simplex.cpp:2309-2315`）。
- (iii) **发布前残差审计**: 每次成功求解都以原始单位重算 $\|\bar{A}x - \bar{b}\|_\infty$ 、界违例与人工列活动度，超过 `escalation_residual_tol`（默认 10^{-6} ）乘以尺度则拒绝发布（`sf_solution_residual_acceptable`, `dual_simplex.cpp:2415-2470`；LPModel 空间版本 `lp_solution_residual_acceptable`, `dual_simplex.cpp:1812-1846`，含 10^{19} 哨兵污染防护）。HiGHS 委托路径同样被审计（`audit_vendored_highs_sf_result`, `dual_simplex.cpp:682-754`）。

2.9 HiGHS 委托路径的数学角色

默认后端下，(2) 被翻译成 HiGHS 模型（人工列固定为 0），由 vendored HiGHS 的单纯形求解（presolve 关、单线程、对偶策略，`dual_simplex.cpp:1354-1374`），结果（基、简约成本、目标）翻译回原生约定并做上节审计。基提示经 `native_sf_basis_to_highs` 转换（`dual_simplex.cpp:558-599`）；持久化路径 `resolve_same_structure` 在结构不变（`structure_id` 匹配）时只增量改界/右端并复用 HiGHS 实例（`dual_simplex.cpp:1095-1210`）。Farkas 射线由 HiGHS `getDualRay` 取得并按公共约定归一化（`dual_simplex.cpp:1420-1489`）。

3 数据结构评估

本节逐项评估关键数据结构，并给出 $m = n = 10^7$ 、 $\text{nnz}(\bar{A}) = 10^8$ （下称“基准规模”）下的内存账目。所有条目为 (V)。

3.1 标准形 StandardFormLP: 矩阵存三份的源头

`StandardFormLP`（`include/mipsolvers/engine/kernel/lp_kernel/dual_simplex.hpp:273-313`）同时持有：

- A: 列主 Eigen 稀疏矩阵 (CSC), 索引类型为 Eigen 默认 `StorageIndex = int` (32 位);
- A_row: 同一矩阵的行主副本 (`dual_simplex.hpp:278`, 注释自述用途为“GMI 的行访问”; 构建于 `dual_simplex_api.cpp:206,449,699`);
- 此外, `HFactorBackend::factorize` 在每次 LU 重构时把整个 $\bar{A}$ 再拷贝一份到 `a_start/a_index/a_value` (`linear_algebra/hfactor_backend.cpp:112-131,48-50`), 且先无条件构造 `Eigen::SparseMatrix<double> Acn(A)` 临时副本 (`hfactor_backend.cpp:112`)——重构瞬间矩阵共存在 4 份。

每项非零元在 CSC 中占 $8 + 4 = 12$ 字节 (值 + 行索引)。基准规模下: $nnz = 10^8 \Rightarrow$ 每份约 1.24 GB (含列指针 $4(n+1)$), 稳态三份约 **3.7 GB**, 重构瞬间约 4.9 GB。对比: HiGHS 内部矩阵只存一份 (列主), 行访问需求由按需构建的 `AR_matrix` 承担且可与列主共享值数组。

发现 1 (D1: 冗余矩阵存储 (严重)). 行主副本 `A_row` 是持久成员而非按需视图; 在 $B \& C$ 中每个被保留的 `SimplexResult` 还持有 `form` 的完整拷贝 (`dual_simplex.cpp:979,1406` 的 `out.form = sf`; `SimplexResult::form` 字段见 `dual_simplex.hpp:317`)。B 个存活节点结果 $\Rightarrow$ 2B 份额外矩阵拷贝。在基准规模下, 仅 10 个存活结果即 ≈ 25 GB。这是数据结构层面最重的浪费。

发现 2 (D2: 32 位索引硬顶 (严重)). 全部索引为 32 位 `int`: `State` 的所有向量 (`native_dual/model.hpp:48-120`)、`HFactorBackend` 的 `HighsInt` (其注释自述 “`== int on this build`”, `linear_algebra/hfactor_backend.cpp:119-120`)、Eigen 默认 `StorageIndex`。列/行号 10^7 本身可表示, 但 nnz 经 `int` 累加 (`hfactor_backend.cpp:117` 的 `static_cast<HighsInt>(Acm.nonZeros())`), 以及 `build_standard_form_lp` 的 `triplet` 计数) 在 $nnz > 2^{31} - 1 \approx 2.1 \times 10^9$ 时溢出。基准规模 (10^8) 尚可, 但平均行非零超过约 200 的 $10^7 \times 10^7$ 问题 (LP 松弛加割后常见) 即越界。维度本身 $> 2^{31}$ 则直接不可用。

3.2 迭代状态 State: 稠密向量组 (合理但有账目)

`State` (`native_dual/model.hpp:48-120`) 持有 `bounds.lower/upper`、`cost`、`original_cost`、`cost_perturbation`、`cost_shift`、`dual_phase_one_anchor` (各 n)、`x_basic` (m)、`reduced_costs` (n) 共 9 个稠密 `Eigen::VectorXd`, 加 `edge_weight` (m 个 `double`) 与若干 `char` 数组。修订单单纯形中稠密 $O(m+n)$ 状态是标准做法 (HiGHS 同), 账目为: $9 \times 8 \times 10^7 \text{ B} \approx 0.72 \text{ GB}$, 可接受。但注意 `cost_perturbation/cost_shift` 与 `original_cost` 三个 n 维向量仅为稳定化簿记, 可用稀疏增量表替代 (省约 0.24 GB, 非致命)。

3.3 稀疏向量 IndexedVector 与打包求解

`IndexedVector` (`native_dual/model.hpp:133-188`) 是 (index, value) 打包稀疏向量, 坐标查询用懒惰构建的开放寻址哈希 (2 倍支持集大小的槽表, Fibonacci 散列, `native_dual/model.hpp:148-180`)。热路径 (`CHUZR/PRICE/FTRAN/DSE`) 全程打包: `HFactorBackend::ftran_indexed/btran_indexed` (`hfactor_backend.cpp:614-717`) 直接驱动 `HFactor` 的超稀疏求解并只导出非零支持。这是合格的超稀疏设计, 与 HiGHS 的 `HVector`

同构。两点保留：(a) `multiply_AT_indexed` 的邮戳累加器是 `thread_local` 稠密数组（8 + 4 字节/列，`native_dual/pricing.cpp:121-138`），基准规模下每线程约 120 MB，连同 `marks`、`primal_delta` 等同类缓冲（`native_dual/pricing.cpp:682-697`、`native_dual/solver.cpp:271-284,476-491,646-656`），每线程常驻约 0.5 GB——线程数乘此数是并行 B&C 的隐性内存税；(b) `dot_error_bound` 对候选列逐系数调用 `row_ep.at()` 哈希查询（`native_dual/pricing.cpp:87-105`），缓存不友好；代码已用 `dense_row_ep` 散射缓解（`native_dual/pricing.cpp:336-356`），属已知折中。

3.4 CHUZR 堆与原始定价堆：合格

对偶离基堆为懒惰版本化二叉堆（`native_dual/model.hpp:49-53`、`native_dual/pricing.cpp:27-85`）：过期条目到堆顶才丢弃，重建触发于稠密变更或条目数 $> 4m$ （`native_dual/pricing.cpp:206-210`）。每次主元仅刷新实际改变的行（`refresh_leaving_heap`，`native_dual/pricing.cpp:181-211`）。原始进入堆带位置索引，支持 $O(\log)$ 删除/刷新（`native_dual/primal.cpp:151-250`）。两者都是正确的工程选择，无占位成分。

3.5 反循环表：std::map 的不当选择与无界增长

`cycle_history`、`taboo_changes`、`taboo_rows` 均为 `std::map`（红黑树，`native_dual/model.hpp:116-119`）。三个问题：(a) 每次查询/插入 $O(\log)$ 且节点指针追逐、缓存行为差——键为 128–192 位元组，应使用开放寻址扁平哈希；(b) `cycle_history` 从不删除条目（`native_dual/state.cpp:999-1064` 只有 `try_emplace` 与 `last_seen` 更新），随不同签名数单调增长：每条约 ≥ 80 字节（树节点开销 + 键 + 记录）， 10^6 个不同签名即 ≥ 80 MB， 10^7 次主元的退化问题可积累到 GB 级；(c) 塔布寿命 $2m + 1$ （`native_dual/state.cpp:976,1053`）在 $m = 10^7$ 时等于“永不失效”，塔布表只增不减。这是真实实现，但数据结构选型与生命周期管理在基准规模下失效。

3.6 LU/FT 后端 HFactorBackend：真实但带全量拷贝

包装层（`linear_algebra/hfactor_backend.cpp`）是真实的：`factorize` 驱动 vendored `HFactor::build`（Markowitz 型 LU + 阈值主元，`hfactor_backend.cpp:144-174`）；`factorize_with_logicals` 实现 HiGHS 式秩修复（缺主元行换逻辑列后重因子化，`hfactor_backend.cpp:231-373`）；FTRAN/BTRAN 支持超稀疏（ $m \geq 2000$ 启用，可用环境变量调，`hfactor_backend.cpp:29-39,386-428`）；FT 更新用捕获的 `HVector` 打包数据（`update_captured`，`hfactor_backend.cpp:734-766`）。三点缺陷：(a) 前述每次 `factorize` 全量拷贝 $\bar{A}$ (D1)；(b) 非打包 `ftran/btran` 每次求解做 $O(m)$ 的排列重映射与 `memcpy` 导出（`hfactor_backend.cpp:391-398,429-433,505-518,551-552`），这些稠密入口被 `reconstruct`、`basis_inverse_row`、`tableau_row` 使用——每次 INVERT 的重建与每次割生成的行提取都是 $O(m)$ 起步，无法利用解的稀疏性（打包入口无此问题，热路径用的正是打包入口）；(c) 打包/稠密两套求解路径并存（2000 行阈值切换），行为（含数值路径）依赖问题规模，注释自述“小系统强制稠密是”数值更稳”（`hfactor_backend.cpp:24-28`）——同一求解器在不同规模下走不同数值路径，是可复现性隐患。

3.7 遗留与冗余字段

`SimplexResult::basis_inverse` 是稠密 `Eigen::MatrixXd` $m \times m$ 字段 (`dual_simplex.hpp:319`)。当前所有求解路径都将其置空 (`dual_simplex.cpp:1036,1602` 的 `resize(0,0)`; `src/engine/solver/native/milp/bc/legacy/bc_relaxation.cpp:745`)，即从未被填充。它是历史稠密实现的 ABI 残留：字段本身不占内存，但任何恢复填充它的改动在 $m = 10^7$ 下意味着 8×10^{14} 字节 (800 TB) ——必须在文档/类型层面明确废弃。`SimplexOptions` 亦含大量自述为“Legacy ABI field”的死参数 (`use_partial_pricing`、`perturb_degenerate_primal`、`fallback_basis`、`enable_degenerate_frontier_remap` 等, `dual_simplex.hpp:148-160,178-189`) ——设置它们不产生任何效果，属于 API 层面的占位符 (见 §5)。

3.8 基准规模内存账目

表 2: $m = n = 10^7$ 、 $\text{nnz} = 10^8$ 下的稳态内存估算 (I): 由数据结构定义推算, LU 填充按基非零 3–10 倍的经验区间)

组件	渐近量	估算
A (CSC)	$12 \text{ nnz} + 4n$	1.24 GB
A_row (行主副本)	$12 \text{ nnz} + 4m$	1.24 GB
HFactorBackend 的 $\bar{A}$ 拷贝	$12 \text{ nnz} + 4n$	1.24 GB
factorize 临时副本 A_{cm} (重构瞬间)	12 nnz	+1.24 GB (瞬时)
State 稠密向量组	$9 \times 8(n+m)$ 量级	0.72 GB
thread_local 累加/邮戳缓冲 (每线程)	$12n + 12m$ 量级	0.5 GB/线程
LU 因子 $L + U$ (填充 3–10× 基非零)	实例相关	0.4–4 GB
cycle_history (10^6 签名)	$\geq 80 \text{ B/条}$	$\geq 0.08 \text{ GB}$
每个存活的 <code>SimplexResult.form</code>	24 nnz	+2.5 GB/个
合计 (单解、不含结果保留)		$\approx 5\text{--}9 \text{ GB}$

结论：单次求解在 16 GB 机器上勉强可行，但内存效率约为理论下限 (单份矩阵 + 迭代状态 + LU) 的 2.5–3 倍；任何“B&C 节点保留 LP 结果”的用法在基准规模下不可行 (D1)。

4 算法效率评估

记号: $s_\rho = |\text{supp}(\rho)|$ (主行支持), s_δ (方向 $\delta = B^{-1}\bar{A}_q$ 的支持), s_π (行对偶支持), $\bar{\nu}$ (候选列平均非零数), K (两次 INVERT 间主元数, ≤ 200)。

4.1 对偶 Phase II 单次主元：真实的超稀疏实现

表 3 逐项给出 `minor_iteration` (`native_dual/solver.cpp:330-797`) 的成本。热路径没有 $O(m)$ 或 $O(n)$ 的隐式全扫描：所有工作都正比于涉及向量的支持集。这与 HiGHS 对偶单纯形的渐近轮廓一致。

表 3: 单次对偶主元的成本分解 ((V))

步骤	成本	代码位置
CHUZR (堆 + 打包 BTRAN)	$O(\log m + \text{nnz}(\text{BTRAN}))$	pricing.cpp:213-305
PRICE ($\rho = \bar{A}^\top \pi_p$)	$O(\sum_{i \in \text{supp}(\pi_p)} \text{nnz}(\bar{A}_i))$	pricing.cpp:117-179
BFRT 扫描	$O(s_\rho)$, 候选点积界最坏 $O(\bar{\nu})$ /列	pricing.cpp:307-869
FTRAN($\bar{A}_q$)	$O(\text{nnz}(\text{FTRAN}))$	solver.cpp:455-465
BFRT 右端 FTRAN (有翻转时)	额外一次打包 FTRAN	solver.cpp:575-584
DSE 权重 (Goldfarb-Reid)	额外一次打包 FTRAN + $O(s_\delta)$	pricing.cpp:940-999
FT 更新	$O(\text{nnz}(\eta))$	hfactor_backend. cpp:734-766
状态增量 (x_β 、 d 、堆刷新、签名)	$O(s_\delta + s_\rho)$	solver.cpp:642-772
合计	4 次稀疏三角求解 + 1 次稀疏 SpMV + 支持集级线性工作	

两点定量保留: (a) **每次主元 4 次三角求解** (BTRAN、FTRAN、BFRT-FTRAN、DSE-FTRAN) 比 HiGHS 的典型 2-3 次多一次——精确 DSE 的 $\varrho = B^{-1}\pi_p$ 是固有代价 (HiGHS 同样支付), 但 BFRT 右端的额外 FTRAN (solver.cpp:578-583) 在翻转频繁时使常数显著增大; (b) BFRT 候选的 dot_error_bound 对每个候选列遍历其全部非零 (pricing.cpp:87-105), 最坏 $O(s_\rho \cdot \bar{\nu})$; 支配系数短路 (pricing.cpp:396-401, 411-424) 依赖 $\|\bar{A}_j\|_1$ 足够紧, 对退化主行 (s_ρ 大) 可能退化为全量遍历。

4.2 INVERT 与其”验证税”

每次重构 (major_rebuild, native_dual/state.cpp:792-914) 的成本远超 LU 本身:

- (i) HFactorBackend::factorize: LU $O(\text{nnz}(L+U))$ + 全量矩阵拷贝 $O(n + \text{nnz}(\bar{A}))$ (hfactor_backend.cpp:112-131);
- (ii) reconstruct: 稠密 FTRAN + BTRAN + 全矩阵 $O(\text{nnz}(\bar{A}))$ 的 $\bar{A}^\top y$ (native_dual/state.cpp:519-591; multiply_AT 遍历全部 n 列, state.cpp:149-161);
- (iii) correct_canonical_primal_residual (由 reconstruct 调用): 构造完整原始点 $O(n)$ + 长双精度残差 $O(\text{nnz}(\bar{A}))$, 超标时再做一次校验 FTRAN 并重算一遍 (native_dual/state.cpp:466-517) ——即**最多 2 趟**长双精度 SpMV;
- (iv) major_rebuild 再调两次 reconstruct (界侧重分类与成本平移后, state.cpp:806,843,870);
- (v) 最终 audit: 又构造完整原始点 + 一趟长双精度残差 (native_dual/state.cpp:1227-1229, 1199-1271)。

合计每次 INVERT: 1 次 LU + 1 次全矩阵拷贝 + 2 次稠密求解 + 约 **3–4 趟长双精度** $O(\text{nnz}(\bar{A}))$ 残差。长双精度 (x87 80 位) FMA 吞吐约为双精度的 $1/4-1/8$, 故仅验证趟就相当于 ≈ 1 趟双精度 SpMV 的 $12-32$ 倍。以 $\text{nnz} = 10^8$ 、每次 INVERT 间隔 $K = 200$ 主元计, 验证税约为每次主元摊销 $10^8 \times 4 \times 4/200 \approx 8 \times 10^6$ 次双精度等效 FLOP——与一个填充适中的三角求解同量级。这不是占位符 (计算是真实且有数学目的的), 但它是按“ m 小、INVERT 便宜”设计的: 在基准规模下验证税与 LU 本身同为支配成本, §6 将其列为致命项 F2。

4.3 冷启动初始化: 精确 DSE 是单独的灾难

冷启动 (无基提示) 默认执行精确 DSE 权重初始化 (`initialize_cold_edge_weights` → `initialize_exact_edge_weights`, `native_dual/state.cpp:1149-1161, 676-719`): 对每个基行 i 做一次带后向误差验证的 BTRAN ($B^{-T}e_i$), 共 m 次 (`compute_exact_edge_weights`, `native_dual/factor.cpp:628-657`)。每次求解经 `checked_btran` 附带 $O(\text{nnz}(B))$ 的残差验证 (`factor.cpp:424-477, 310-350, 526-542`)。总成本 $\Omega(m \cdot (\text{BTRAN} + \text{nnz}(B)))$ 。 $m = 10^4$ 时已是秒级–分钟级; $m = 10^7$ 、即使每次 BTRAN 仅 10^4 非零, 也是 $\geq 10^{11}$ 次运算的天数级开销, 外加 m 趟 $O(\text{nnz}(B))$ 验证。HiGHS 在大问题上用单位权重 + 递推修正近似初始化精确 DSE, 正是为了避开 m 次求解。环境变量 `MIPSOLVERS_DUAL_PRICING=devex` 可切换到 Devex 避开 (`native_dual/state.cpp:1154-1159`), 但默认策略在基准规模下不可用。此为致命项 F1。

4.4 原始单纯形阶段的成本轮廓

`run_primal_phase` 每次主元: 1 次打包 FTRAN + 1 次打包 BTRAN + 1 次 PRICE + Harris 比测试 $O(s_\delta)$ + 堆刷新 $O(s_\rho \log)$, 轮廓合格。但 PSE 模式初始化对每个非基列做一次 FTRAN (`initialize_primal_pse`, `native_dual/primal.cpp:252-274`): $O(n)$ 次求解, $n = 10^7$ 下绝无可能; Devex (默认) 无此问题。PSE 属于“实现了但仅小维可用”的模式 (见 §5 P4)。

4.5 标准形与边界的增量维护: 合格

`update_standard_form_bounds` (`dual_simplex_api.cpp:855-971`) 对 ≤ 50 个下界变化走逐列增量 ($O(\sum \text{nnz}(\text{变化列}))$), 否则全量 SpMV; `update_standard_form_bounds_incremental` (`dual_simplex_api.cpp:976-1078`) 先完整校验变更序列再应用, 避免兄弟节点陈旧增量污染。`update_standard_form_cost` (`dual_simplex_api.cpp:722-756`) $O(n)$ 保结构换成本。这些是 B&C 节点 LP 的正确工程: 每节点 $O(1)-O(\text{几列})$ 。

4.6 HiGHS 委托路径的成本

非持久化路径每次求解: 完整遍历 $\bar{A}$ 构造 HiGHS CSC (`pass_standard_form_to_highs`, `dual_simplex.cpp:627-680`, $O(n + \text{nnz})$ 拷贝与逐元素 `push_back`) + HiGHS 内部再持有一份——即求解前至少 2 趟全矩阵拷贝。持久化路径 (`resolve_same_structure`) 消除之, 但要求 `structure_id` 匹配与逐元素成本相等校验 ($O(n)$, `dual_simplex.cpp:1130-1139`), 并

有失败时完整快照回滚再 `run()` 一次的代价 (`dual_simplex.cpp:1208,1303-1323`)。结果回填阶段的 `import_optimal_result` 与 `inject_basic_degenerate_col_duals_from_highs` 对每个基本整数/二进制列调用 `getReducedRow` (一次 BTRAN + $O(n)$ 扫描, `dual_simplex.cpp:756-859`) ——整数列占比高时是 $O(n)$ 次 BTRAN 量级, 在基准规模下同样是初始化级灾难, 但该注入服务于 MIP 传播的对偶质量, 属于功能性代价而非错误。

4.7 复杂度小结

热路径 (对偶 Phase II 主元) 渐近合格; 成本集中在三处结构性开销: 每次 INVERT 的全矩阵拷贝与长双精度审计 (F2)、冷启动精确 DSE (F1)、矩阵三份存储带来的带宽压力 (D1)。换言之, 该内核的效率问题不在主循环算法, 而在围绕主循环的验证与初始化机器——这与“看起来复杂”的观感来源一致: 复杂度集中在正确性证据的构造上, 而非求解本身。

5 占位符与缺陷清单

本节回答用户的核心怀疑: “代码内有大量占位符, 很多是虚假的, 只是看起来复杂。”按证据强度分三类。P 类: 字面意义的死代码/虚假参数——存在、精心编写、从不产生效果; Q 类: 数学上不构成其所声称之物的实现缺口; R 类: 真实实现但仅在小维有意义的“规模占位符”。每条附 (V)/(I) 标注。

5.1 P 类: 死代码与虚假参数 (证据确凿)

发现 3 (P1: 约 290 行长双精度“精确验证”机器是零调用死代码 ((V), 最严重)). `BasisFactor::pivot_evidence (native_dual/factor.cpp:138-216)`、`row_solve_consistent (native_dual/factor.cpp:218-308)`、`refine_ftran (native_dual/factor.cpp:623-626)` 三个函数以长双精度构造主元行/列恒等性的严格误差包络 (γ_n 放大、残差包络、算术守卫), 约 290 行, 读起来像是内核数值正确性的核心。全仓库 (`src/` 与 `tests/`) 符号搜索确认: 除自身定义与头文件声明外无任何调用点。主元行/列一致性实际由两处轻量检查承担: 对偶侧的 `BFRT` 后验条件 (`native_dual/solver.cpp:541-572`) 与原始侧的 `primal_pivot_identity_acceptable (native_dual/primal.cpp:652-667, native_dual/primal_pricing.hpp:219-226)`。这正是“只是看起来复杂”的典型: 它们给人留下“每次主元都经严格验证”的印象, 但真实热路径并不使用它们。(保留可能性: 作为未来严格模式的预留。但按现状报告: 死代码。)

发现 4 (P2: `is_artificial` 的 $O(m)$ 线性扫描 ((V))). `native_dual/state.cpp:173-178`: 每次调用线性扫描 `row_to_artificial_col` (长度 m)。同为零调用死代码 (声明于 `state.hpp:27`, 无调用点)。若被用于逐列判定将是 $O(m)$ /次的陷阱; 正确结构是 `aux_col_to_row` 反向表 (已存在于 `StandardFormLP, dual_simplex.hpp:298`)。

发现 5 (P3: `SimplexOptions` 的遗留 ABI 参数字段 ((V))). 以下字段设置后无任何效果, 注释自述“Legacy ABI field”: `allow_cold_start`、`use_partial_pricing`、`perturb_degenerate_primal`、`fallback_basis`、`exact_dse_initialization`、`enable_/suppress_`

`degenerate_frontier_remap`、`escalation_max_level`、`escalation_umfpack_pivot_tolerance`、`escalation_ruiz_rounds`、`escalation_start_level`、`escalation_max_level_sf` (`include/mipsolvers/engine/kernel/lp_kernel/dual_simplex.hpp:137-195`)。调用方若以为 `use_partial_pricing=true` 启用了部分定价、或 `fallback_basis` 提供了失败回退，得到的是静默无效。这是 API 层面的占位符：保留编译兼容，但语义为空。另：`dump_solve_lp_counters()` 是显式 no-op (`dual_simplex.cpp:2000`)。

发现 6 (P4: `SimplexResult::basis_inverse` 稠密逆矩阵字段((V))). `Eigen::MatrixXd` 类型的 $m \times m$ 稠密字段 (`dual_simplex.hpp:319`), 所有路径置空 (`dual_simplex.cpp:1036,1602`)。结构层面的“我们好像能提供 B^{-1} ”的假象；真实接口是 `BasisOps::basis_inverse_row` (按需单行)。见 §3。

5.2 Q 类：数学缺口（声称与实现不符）

发现 7 (Q1: 无原始无界证书，无界被报为数值失败 ((V), 功能缺陷)). `Status` 枚举没有 `Unbounded` (`native_dual/native_dual_core.hpp:13-22`)。原始单纯形 Harris 比测试得不到有限步长（数学上即发现无界射线 $\delta = B^{-1}\bar{A}_q$, $\delta \leq 0$ 且 $d_q > 0$ ）时，代码返回 `NumericalFailure` 并附言“an unbounded certificate is required” (`native_dual/primal.cpp:525-541`)——即代码承认此处需要无界证书但未实现。对偶侧同理：对偶 Phase I 以 `DualInfeasibleStart` 结束时并不产出原始无界射线 (`native_dual/solver.cpp:855-863`)。对偶单纯形在 Phase II 中理论上不会遇到原始无界（对偶可行下目标有界），所以日常路径不触发；但作为通用 LP 求解器，无界判定只能依赖 HiGHS 委托路径 (`kUnbounded` 标签, `dual_simplex.cpp:541-543`)。原生内核的证书体系 (§2.8) 是单侧的：只有不可行证书，没有无界证书。

发现 8 (Q2: 后向误差验证的范数在 FT 更新后陈旧 ((V), 严格性缺口)). `backward_error_acceptable` 的误差限为 $256\epsilon \max(1, \|B\|_\infty \|x\|_\infty + \|r\|_\infty)$ (`native_dual/factor.cpp:526-542`)，其中范数 `norm_B_inf` 只在 `rebuild/rebind_A` 时重算 (`factor.cpp:79,401-422,711`)。稠密 `update` 路径有 $O(m)$ 的 `max_element` 范数维护 (`factor.cpp:115-120`)，但热路径走 `update_indexed` (`factor.cpp:125-136`)，它不更新范数。因此第 k 次 FT 更新后的校验求解所用误差包络基于 k 步之前的基： $\|B^{(0)}\|_\infty$ 并不控制 $\|B^{(k)}\|_\infty$ 。缓解事实：打包求解（热路径）根本不做后向误差验证 (`indexed_ftran/indexed_btran` 只查有限性, `factor.cpp:591-621`)，且每次 `INVERT` 后范数重新同步——所以这不是“证书造假”，而是验证包络在两个 `INVERT` 之间逐渐失去严格性。报告因为它与代码注释自述的“每次求解都经后向误差验证”的形象不符。

发现 9 (Q3: `update` 的 $O(m)$ 范数维护 ((V), 潜在陷阱)). 若任何未来调用方使用稠密 `BasisFactor::update` (当前无调用点, `factor.cpp:85-123`)，每次主元将支付两次 $O(m)$ 的 `max_element` (`factor.cpp:115-120`)。当前无影响；列为陷阱。

5.3 R 类：真实但仅小维有意义的实现

发现 10 (R1: 精确 DSE 初始化 ((V))). 见 §4.3: m 次带验证 *BTRAN*。数学真实, $m = 10^7$ 不可用。

发现 11 (R2: PSE 定价初始化 ((V))). *initialize_primal_pse* 对每个非基列一次 *FTRAN* (*native_dual/primal.cpp:252-274*): $O(n)$ 次求解。默认关闭 (*MIPSOLVERS_PRIMAL_DEVEX* 默认 *Devel*, *native_dual/primal.cpp:108-116*), 但 *PSE* 模式在基准规模下是装饰性的。

发现 12 (R3: 每次 *INVERT* 的长双精度审计趟 ((V))). 见 §4.2: 3-4 趟 $O(\text{nnz}(\bar{A}))$ 长双精度残差。数学真实且动机正当 (*fail-closed* 发布), 常数在基准规模下致命。

发现 13 (R4: *compute_exact_edge_weights* 的逐行单位向量 *BTRAN* ((V))). *native_dual/factor.cpp:628-657*: m 次独立 *BTRAN* 而非批量三角求解 (无法利用共同的 L/U 结构做块化处理)。真实但效率为理论下限的数倍。

发现 14 (R5: *inject_basic_degenerate_col_duals_from_highs* ((V))). *HiGHS* 路径下对每个基本整数列一次 *getReducedRow* + $O(n)$ 扫描 (*dual_simplex.cpp:756-859*)。功能真实 (改善 *MIP* 传播对偶), 全整数大模型下为 $O(n)$ 次 *BTRAN*。

6 千万维可扩展性结论

6.1 致命项排序

在 $m = n = 10^7$ 、 $\text{nnz}(\bar{A}) = 10^8$ 的通用 LP 上, 按”是否单独即使求解不可行”排序:

表 4: 致命项与定量理由

#	致命项	定量理由
F1	冷启动精确 DSE: m 次带验证 <i>BTRAN</i> (§4.3)	$\geq 10^{11}$ 次运算 (天数级), 每次冷启动必付
F2	每次 <i>INVERT</i> 的 3-4 趟长双精度 $O(\text{nnz})$ 审计 + 全矩阵拷贝 (§4.2)	每 <i>INVERT</i> $\approx 4 \times 10^8$ 长双精度 FLOP + 1.2 GB 拷贝; 10^4 次 <i>INVERT</i> 即 $\approx 4 \times 10^{12}$ 等效双精度 FLOP
F3	矩阵三份常驻 + 每次 <i>INVERT</i> 第四份瞬时拷贝 (D1)	稳态 3.7 GB、重构瞬间 4.9 GB, 仅为矩阵本身; 叠加 LU 与状态后 16 GB 机器无余量做 B&C
F4	32 位索引 (D2)	$\text{nnz} > 2.1 \times 10^9$ 或维度 $> 2^{31}$ 直接溢出; 加割后的 10^7 维问题可触及
F5	<i>cycle_history</i> /塔布 <i>std::map</i> 无界增长	高退化 10^7 维问题的 10^6 - 10^7 次主元产生 GB 级红黑树, 且塔布寿命 $2m+1$ 等于永久

#	致命项	定量理由
F6	非打包 FTRAN/BTRAN 的 $O(m)$ 稠密导出	每次 INVERT 重建与每次割行提取 $O(m)$ 起步； 10^4 次 INVERT 即 10^{11} 次内存访问

此外有两个正确性边界在基准规模下同样需要重审：FT 更新上限 $K \leq 200$ (solver.cpp:976-986) 意味着 10^6 次主元需 5×10^3 次 LU 重构，每次重构是 10^7 阶 Markowitz LU——若基非零为 5×10^7 ，单次重构在现有实现（含 F2 验证税）下为分钟级，总时间以周计；以及 Q2 的验证包络随更新次数失去严格性。

6.2 必须修改的清单（按投入产出排序）

- M1. 用近似 DSE 初始化替换精确初始化：单位权重 + 递推修正（HiGHS 默认策略），保留精确路径为选项。消除 F1。改动集中于 `initialize_cold_edge_weights` (state.cpp:1149-1161)。
- M2. 审计趟降频与降精度：长双精度残差改为双精度 + 仅每 k 次 INVERT 执行完整审计（保留 fail-closed 发布审计不动——那是结果正确性的最后防线）。缓解 F2。
- M3. 矩阵单份化：A_row 改为按需构建/共享值数组的行视图；HFactorBackend 改为引用外部 CSC 缓冲而非拷贝；SimplexResult 不持有 form（或持共享指针）。消除 F3。
- M4. 索引宽度参数化：Eigen StorageIndex 与 HighsInt 提升到 64 位（至少 nnz 计数与列指针）。消除 F4。
- M5. 循环防护换扁平哈希 + 容量上限：开放寻址表替换 `std::map`，`cycle_history` 加 LRU 上限，塔布寿命与 m 解耦。消除 F5。
- M6. 删除或接线 P1 死代码：要么把 `pivot_evidence/row_solve_consistent` 接入严格验证模式（替代 Q2 的陈旧包络），要么删除——当前形态只产生虚假的安全感。
- M7. 实现无界射线证书：Harris 比测试无有限步长时输出 δ 射线与 d_q (Q1)，否则原生内核不能作为通用 LP 求解器独立发布无界结论。
- M8. API 清理：删除或显式废弃 P3/P4 的遗留字段；`max_iter` 默认 2000 (dual_simplex.hpp:130) 对通用 LP 过低，应由调用方按问题规模设定。

6.3 最终结论

`native_dual/` 不是“虚假的复杂”：它是一次认真的、防御性极强的有界变量修订对偶单纯形实现，其数值机器（打包超稀疏求解、增量循环签名、懒惰堆、Farkas 区间证书、fail-closed 发布审计）在 $m \lesssim 10^4$ 的 B&C 节点 LP 上是合格的，部分设计（Zobrist 增量签名、版本化堆）达到先进实现水准。但用户的怀疑在三个可证实的意义上成立：(i) 存在约 290 行精心编写却零调用的“严格验证”死代码（P1），它是“看起来复杂”的直接来源；(ii) API 表面保留

了一打设置后无任何效果的遗留参数 (P3/P4); (iii) 内核的成本重心不在求解而在验证与初始化——精确 DSE 初始化 (F1)、每次 INVERT 的长双精度审计 (F2)、三份矩阵存储 (F3) 每一项都真实地执行着在千万维下不可能完成的计算。当前形态下, 该内核可用于中小规模 B&C 节点 LP; 作为面向 10^7 维通用 LP 的求解内核, 需要完成 M1-M5 后才具备讨论资格。

A 审计方法与可复核性

本报告全部论断基于报告日工作区的静态阅读。复核方式:

- 死代码判定 (P1/P2): 对 `pivot_evidence`、`row_solve_consistent`、`refine_ftran`、`is_artificial` 在 `src/` 与 `tests/` 全树做符号引用搜索, 除定义与声明外无命中。
- 默认后端判定: `SimplexOptions::lp_kernel_backend` 默认值见 `include/mipsolvers/engine/kernel/lp_kernel/dual_simplex.hpp:173`; 原生内核的显式选择点见 `tests/test_dual_simplex.cpp:141`、`benchmark/native_kernel_comparison.cpp:308` 等。
- 复杂度陈述均可在所引行号的循环结构上直接验证。

本审计未执行运行时剖析; F1-F6 的定量估算为按数据结构定义与循环次数的下界推算 (I), 实际常数可能更差 (缓存未命中、长双精度吞吐) 但不会更好。